

A Custom Autograd Function, Worked Out by Hand

Supplying your own forward and backward for ClampedReLU (ReLU6)

PyTorch already knows how to differentiate `sin`, `exp`, `matmul` and hundreds of other primitives. The moment you invent an operation it has never seen, you must hand it two things: the *forward* formula and the *local derivative* the chain rule needs. This document picks one such operation — a clamped ReLU capped at 6 (the “ReLU6” used in MobileNets) — and works the entire forward/backward pass out by hand on the vector $x = [-2, 0.5, 3, 7]$. Every number below was produced by the NumPy/PyTorch program in Appendix A, so the arithmetic is exact and self-consistent.

What this document does. It defines `ClampedReLU` as a `torch.autograd.Function`, computes its forward output, derives the backward Jacobian as a diagonal 0/1 mask, multiplies by two different upstream gradients, and then *verifies* the analytic backward against a finite-difference gradient — including showing exactly how that numerical check misleads you at the two kinks.

How to read this. Each step does one thing and gets a coloured box isolating the idea on the cleanest numbers. If you only remember one sentence, make it the Intuition box in Step 3: *a custom backward is just the local Jacobian, and the chain rule does the rest.*

Concepts covered, in order: why you would ever write a custom Function · the forward map $f(x) = \text{clamp}(x, 0, 6)$ · `ctx.save_for_backward` · the elementwise Jacobian and the 0/1 gradient mask · the chain rule downstream $= g \odot \text{mask}$ for two upstream vectors · the subgradient choice at the kinks $x = 0$ and $x = \text{cap}$ · finite-difference gradient checking and where it lies · surrogate gradients and the straight-through estimator · `torch.autograd.gradcheck` in float64.

1 Why supply your own forward and backward

Autograd is a bookkeeper, not a mathematician. It records the primitive operations you call and replays their *known* derivatives backwards. So you only need to write a custom `Function` when one of the following is true:

Reason	What goes wrong without a custom <code>Function</code>
Non-differentiable op	A hard threshold, rounding, sampling, or <code>argmax</code> has a zero/undefined derivative; autograd cannot push a useful signal through it.
Non-standard math	You wrap an external solver, a C/CUDA kernel, or a special function PyTorch has no rule for.
Speed / fusion	You can hand-write a single backward that is cheaper than differentiating a long chain of primitives element-wise.
Surrogate gradient	You deliberately want a <i>different</i> backward than the true one — e.g. a straight-through estimator for a quantiser.

Our example, **ClampedReLU** with `cap = 6`, is the first kind: it is continuous but has two *kinks* where the true derivative does not exist, so we must tell autograd which subgradient to use. The forward and backward are:

$$f(x) = \text{clamp}(x, 0, 6) = \min(\max(x, 0), 6)$$

$$\frac{df}{dx} = \begin{cases} 1 & 0 < x < 6 \\ 0 & \text{otherwise.} \end{cases}$$

When this is the right choice

Reach for `torch.autograd.Function` when the operation is genuinely new or non-differentiable, or when you want to override the gradient on purpose. If your operation is just a composition of existing differentiable ops (a few `matmuls` and a `relu`), do *nothing* — autograd already differentiates it correctly and a hand-written backward only adds bugs.

2 Step 1 — The forward pass

We work the whole document on one vector, chosen so every branch of the clamp is exercised: one value below 0, one strictly inside $(0, 6)$, one more inside, and one above the cap.

$$x = [-2 \quad 0.5 \quad 3 \quad 7], \quad \text{cap} = 6.$$

Apply $f(x) = \text{clamp}(x, 0, 6)$ elementwise. Below 0 snaps up to 0; above 6 snaps down to 6; in between, the value passes through unchanged:

$$f(x) = \begin{bmatrix} \text{clamp}(-2) \\ \text{clamp}(0.5) \\ \text{clamp}(3) \\ \text{clamp}(7) \end{bmatrix} = \begin{bmatrix} 0 & 0.5 & 3 & 6 \end{bmatrix}.$$

The first entry was clamped *up* (the $\max(\cdot, 0)$ floor), the last clamped *down* (the $\min(\cdot, 6)$ ceiling), and the two middle entries are untouched.

Mini-example — this operation in isolation

The clamp is two one-sided gates composed. $\max(x, 0)$ turns $[-2, 0.5, 3, 7]$ into $[0, 0.5, 3, 7]$ (only the negative moves); then $\min(\cdot, 6)$ turns that into $[0, 0.5, 3, 6]$ (only the 7 moves). Same answer, and it makes the two kinks visible: the floor acts at $x = 0$, the ceiling at $x = 6$.

3 Step 2 — `ctx.save_for_backward`: what the backward needs

The backward pass needs to know *which* entries were in the active zone $0 < x < 6$. That information lives in the original input x , not in the output $f(x)$ — so during forward we stash x on the context object:

`ctx.save_for_backward(x)` $\implies$ `x, = ctx.saved_tensors`

`cap` is a plain Python float, not a tensor, so it rides along as `ctx.cap = cap` rather than through `save_for_backward`. This matters for the return contract of `backward`, which we meet in Step 4.

Intuition

Think of `ctx` as a sealed envelope the forward pass mails to its own future backward pass. Save only what the derivative actually requires — here, the raw input x — because everything saved is held in memory until the backward runs. Saving $f(x)$ instead would be a bug: the clamped output $[0, 0.5, 3, 6]$ cannot tell apart a genuine 0 from a clamped-down value, so you could not recover the mask.

4 Step 3 — The backward Jacobian: a 0/1 mask

Because f acts elementwise, its Jacobian $J = \partial f / \partial x$ is *diagonal*: entry i of the output depends only on entry i of the input. Each diagonal term is the scalar derivative of the clamp,

$$J_{ii} = f'(x_i) = \mathbb{I}[0 < x_i < 6] = \begin{cases} 1 & 0 < x_i < 6 \\ 0 & \text{else.} \end{cases}$$

Evaluating the indicator on $x = [-2, 0.5, 3, 7]$ gives the **gradient mask**

$$m = \begin{bmatrix} \mathbb{I}[0 < -2 < 6] \\ \mathbb{I}[0 < 0.5 < 6] \\ \mathbb{I}[0 < 3 < 6] \\ \mathbb{I}[0 < 7 < 6] \end{bmatrix} = \boxed{\begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}}, \quad J = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}.$$

The two outer entries were clamped (saturated), so a small wiggle in their input does *not* move the output — their local derivative is 0. The two inner entries pass their input straight through, so their derivative is 1.

Intuition

This 0/1 mask *is* the entire job of **backward**: you are supplying the local Jacobian the chain rule will multiply by. Autograd handles the global graph; you only answer “if the input here wiggles by δ , how much does my output wiggle?” For a saturated clamp the honest answer is “not at all”, hence 0.

5 Step 4 — The chain rule: $\text{downstream} = g \odot m$

In backprop, **backward** receives the *upstream* gradient $g = \partial L / \partial f$ flowing back from the loss, and must return the *downstream* gradient $\partial L / \partial x$. By the chain rule with a diagonal Jacobian, this is just an elementwise product:

$$\frac{\partial L}{\partial x} = J^\top g = m \odot g.$$

Case A — upstream all ones ($g = [1, 1, 1, 1]$, which is what `y.sum().backward()` sends, since $\partial(\sum_i f_i) / \partial f_j = 1$):

$$\frac{\partial L}{\partial x} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 1 \\ 1 \\ 1 \\ 1 \end{bmatrix} = \boxed{\begin{bmatrix} 0 & 1 & 1 & 0 \end{bmatrix}}.$$

Only the two active entries receive gradient; the saturated entries are blocked.

Case B — a non-trivial upstream $g = [0.5, 2, -1, 3]$:

$$\frac{\partial L}{\partial x} = \begin{bmatrix} 0 \\ 1 \\ 1 \\ 0 \end{bmatrix} \odot \begin{bmatrix} 0.5 \\ 2 \\ -1 \\ 3 \end{bmatrix} = \boxed{\begin{bmatrix} 0 & 2 & -1 & 0 \end{bmatrix}}.$$

The mask gates: wherever it is 0 the upstream value is annihilated regardless of its magnitude or sign; wherever it is 1 the upstream value passes through unchanged. This is why saturated activations “kill” gradients — a known cause of dead units.

Watch out

backward must return *one gradient per input that forward received*, in the same order. `forward(ctx, x, cap)` took two arguments, so **backward** returns `(grad_input, None)`: a tensor for x and `None` for `cap`, because `cap` is not a differentiable tensor. Returning a single tensor (forgetting the `None`) raises a count-mismatch error at `.backward()` time.

6 Step 5 — The kinks: choosing a subgradient

At $x = 0$ and $x = 6$ the function f has a *corner*: the slope is 0 on one side and 1 on the other, so the true derivative does not exist. Any value in $[0, 1]$ is a valid *subgradient* there. We must pick one, and our convention $\mathbb{K}[0 < x < 6]$ (strict inequalities) chooses 0 at both kinks.

This is a deliberate choice, not a fact. The course’s own code uses the same strict convention; some libraries instead let the boundary pass gradient. For ReLU6 the choice is harmless because a kink is hit with probability zero in floating point, but you should *know* you made it.

Watch out

A finite-difference check straddles the kink and reports the *averaged* slope, not your chosen subgradient. At $x = 0$ with $\varepsilon = 0.1$,

$$\frac{f(0.1) - f(-0.1)}{2\varepsilon} = \frac{0.1 - 0}{0.2} = 0.5,$$

and identically 0.5 at $x = 6$. So a numerical check at a kink will “disagree” with your backward by 0.5 even though your backward is correct by convention. *Always gradient-check at interior points, never on a corner.*

7 Step 6 — Verifying with a numerical gradient

To trust a hand-written backward, compare it to a finite-difference (FD) estimate of the same gradient. For the scalar loss $L = \sum_i f(x_i)$ the central difference in coordinate i is

$$\left(\frac{\partial L}{\partial x}\right)_i \approx \frac{L(x + \varepsilon e_i) - L(x - \varepsilon e_i)}{2\varepsilon}.$$

Run at the safe interior point $x = 3$ (where $0 < 3 < 6$), the estimate is exact for every ε we tried, because f is locally the identity there:

$$\varepsilon \in \{10^{-1}, 10^{-3}, 10^{-5}\} \implies \frac{f(3 + \varepsilon) - f(3 - \varepsilon)}{2\varepsilon} = 1.000 = f'(3).$$

Across the whole vector (interior coordinates only contribute), the FD gradient of the sum matches the analytic mask to floating-point noise:

$$\text{FD} = [0 \quad 1 \quad 1 \quad 0], \quad m = [0 \quad 1 \quad 1 \quad 0], \quad \max_i |\text{FD}_i - m_i| = 3.786 \times 10^{-11}.$$

PyTorch packages this comparison as `torch.autograd.gradcheck`. Run in `float64` on inputs chosen away from the kinks ($[-2, 0.5, 3, 5]$), it returns `True`: the custom backward is numerically correct.

Watch out

`gradcheck` *requires* `float64` inputs. In `float32` the 2ε denominator amplifies rounding error and the check fails spuriously — the same effect the course shows for $f(x) = x^3$, where an `float32` FD gradient at $x = 3$ overshoots the true 27 by about 0.084. Cast to `double` before calling `gradcheck`.

8 Step 7 — Surrogate gradients and the straight-through estimator

Sometimes the true derivative is useless and you *override* it on purpose. The canonical case is the **straight-through estimator** (STE): the forward applies a hard, non-differentiable map (round, sign, quantise) whose true gradient is 0 almost everywhere, so a faithful backward would block all learning. The STE instead pretends the backward is the identity:

$$\text{forward: } y = \text{round}(x), \quad \text{backward (surrogate): } \frac{\partial L}{\partial x} \leftarrow g \quad (\text{as if } \frac{dy}{dx} = 1).$$

Concretely, if you replaced our mask with the all-ones surrogate $m_{\text{STE}} = [1, 1, 1, 1]$, the upstream $g = [0.5, 2, -1, 3]$ would pass through *untouched* as $[0.5, 2, -1, 3]$ instead of being gated to $[0, 2, -1, 0]$. That is the whole trick: a `Function` lets you decouple the forward you want to compute from the backward you want to learn with.

When this is the right choice

Use a surrogate gradient when the honest gradient is zero or undefined but you still need a learning signal through the op — quantisation-aware training, binary/spiking networks, discrete sampling. Use the *true* subgradient (our 0/1 mask) when the op is genuinely the activation you intend, as with ReLU6. Both are one-line changes inside the same **backward**.

9 ClampedReLU at a glance

i	input x_i	forward $f(x_i)$	mask m_i	downstream for g_2
0	-2	0	0	0
1	0.5	0.5	1	2
2	3	3	1	-1
3	7	6	0	0
column sum		9.5	—	1

The forward column sums to 9.5 (the scalar loss $\sum_i f_i$), and with upstream $g_2 = [0.5, 2, -1, 3]$ the downstream gradient is $[0, 2, -1, 0]$. Read the table left-to-right and you have traced one full forward and backward pass by hand.

10 Equation cheat-sheet

Forward	$f(x) = \text{clamp}(x, 0, \text{cap}) = \min(\max(x, 0), \text{cap})$
Local derivative	$f'(x) = \mathbb{K}[0 < x < \text{cap}] \in \{0, 1\}$
Jacobian	$J = \text{diag}(m), \quad m_i = \mathbb{K}[0 < x_i < \text{cap}]$
Chain rule (backward)	$\frac{\partial L}{\partial x} = J^\top g = m \odot g$
<code>sum().backward()</code>	upstream $g = \mathbf{1} \Rightarrow \partial L / \partial x = m$
Return contract	one grad per forward input: <code>(grad_input, None)</code>
Kink subgradient	strict $< \Rightarrow$ chooses 0 at $x = 0$ and $x = \text{cap}$
Central FD	$\partial_i L \approx (L(x + \varepsilon e_i) - L(x - \varepsilon e_i)) / 2\varepsilon$
STE surrogate	replace m by $\mathbf{1}$: $\partial L / \partial x = g$

Appendix A Reference implementation

Every number above is reproducible from the following program. The `gradcheck` runs in `float64` on inputs chosen away from the kinks.

```
import torch

class ClampedReLU(torch.autograd.Function):
    """ReLU6: f(x) = clamp(x, 0, cap). grad = 1 if 0 < x < cap else 0."""
    @staticmethod
    def forward(ctx, x, cap=6.0):
        ctx.save_for_backward(x)      # save input for backward
        ctx.cap = cap                  # cap is a float, not a tensor
        return x.clamp(0.0, cap)

    @staticmethod
    def backward(ctx, grad_output):
        x, = ctx.saved_tensors
        cap = ctx.cap
        mask = ((x > 0) & (x < cap)).to(grad_output.dtype) # 0/1 Jacobian diag
        grad_input = grad_output * mask                     # chain rule
        return grad_input, None                             # one grad per forward input (None for cap)

# --- forward + backward on the worked vector ---
x = torch.tensor([-2.0, 0.5, 3.0, 7.0], requires_grad=True)
y = ClampedReLU.apply(x, 6.0)
print("forward:", y.detach())          # [0.0, 0.5, 3.0, 6.0]

y.sum().backward()                    # upstream g = [1,1,1,1]
print("grad (g=ones):", x.grad)       # [0., 1., 1., 0.]

# a non-trivial upstream gradient g2 = [0.5, 2, -1, 3] -> [0, 2, -1, 0]
x2 = torch.tensor([-2.0, 0.5, 3.0, 7.0], requires_grad=True)
y2 = ClampedReLU.apply(x2, 6.0)
y2.backward(torch.tensor([0.5, 2.0, -1.0, 3.0]))
print("grad (g2):", x2.grad)          # [0., 2., -1., 0.]

# --- finite-difference check of the sum-loss gradient ---
def loss(t): return ClampedReLU.apply(t, 6.0).sum()
xv = torch.tensor([-2.0, 0.5, 3.0, 7.0], dtype=torch.float64)
eps = 1e-5
fd = torch.zeros_like(xv)
for i in range(len(xv)):
    xp = xv.clone(); xp[i] += eps
    xm = xv.clone(); xm[i] -= eps
    fd[i] = (loss(xp) - loss(xm)) / (2 * eps)
print("FD grad of sum:", fd)          # [0., 1., 1., 0.] (~3.8e-11 error)

# --- official gradcheck in float64, at interior points (avoid kinks) ---
xc = torch.tensor([-2.0, 0.5, 3.0, 5.0], dtype=torch.float64, requires_grad=True)
ok = torch.autograd.gradcheck(lambda t: ClampedReLU.apply(t, 6.0), (xc,),
                              eps=1e-6, atol=1e-6)
print("gradcheck passed:", ok)       # True

# --- a kink misleads finite differences (do NOT check here) ---
def f(v): return float(torch.tensor(v).clamp(0.0, 6.0))
for xk in (0.0, 6.0):
    e = 0.1
```

```
print(f"FD at kink x={xk}: {(f(xk+e)-f(xk-e))/(2*e):.4f}") # 0.5, 0.5
```

— end of document —